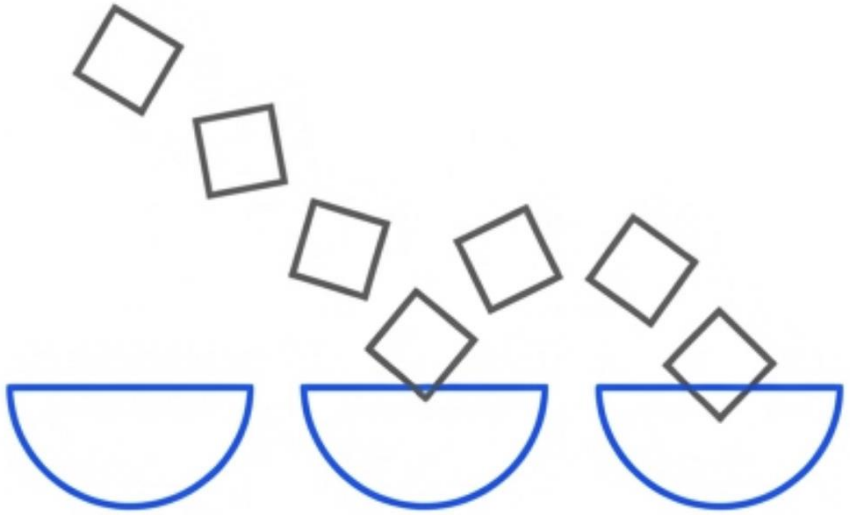
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Algoritmos de ordenamiento:

BucketSort



Objetivos

- Identificar las condiciones matemáticas necesarias (distribución uniforme en $[0,1)$) para que BucketSort evite el límite asintótico inferior de $\Omega(n \log n)$ característico de los algoritmos basados en comparaciones.

- Demostrar formalmente, mediante el uso de variables indicadoras aleatorias, por qué el tiempo de ejecución esperado en el caso promedio es $O(n)$.

- Escribir una implementación robusta de BucketSort en C++ moderno, gestionando correctamente la proyección de valores reales hacia índices discretos de casilleros (buckets).

- Diagnosticar las deficiencias de rendimiento provocadas por los fallos de caché (cache misses) cuando se emplean estructuras de datos no contiguas (como listas enlazadas o múltiples vectores dinámicos pequeños) para representar los casilleros.

- Rediseñar la estructura de memoria subyacente del algoritmo empleando un enfoque orientado a datos (Data-Oriented Design) que garantice una alta localidad espacial y maximice el rendimiento en arquitecturas de CPU modernas.

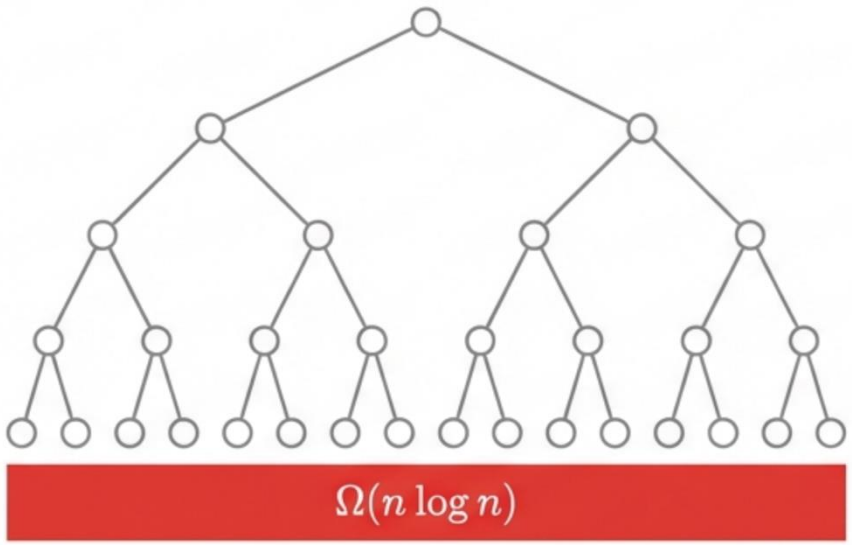
Motivación

Todo algoritmo de ordenamiento basado estrictamente en comparaciones (como MergeSort, HeapSort o QuickSort) está restringido por el modelo de árbol de decisión. Como se demuestra en la teoría fundamental de algoritmos, cualquier árbol de decisión que ordene n elementos debe tener altura $\Omega(n \log n)$ *(Cormen, Capítulo 8, Sección 8.1)*. Esta es una barrera matemática infranqueable si nuestra única operación fundamental es la comparación bidireccional entre elementos.

La motivación detrás de **BucketSort** es romper esta barrera asintótica mediante el uso de información intrínseca de los datos.

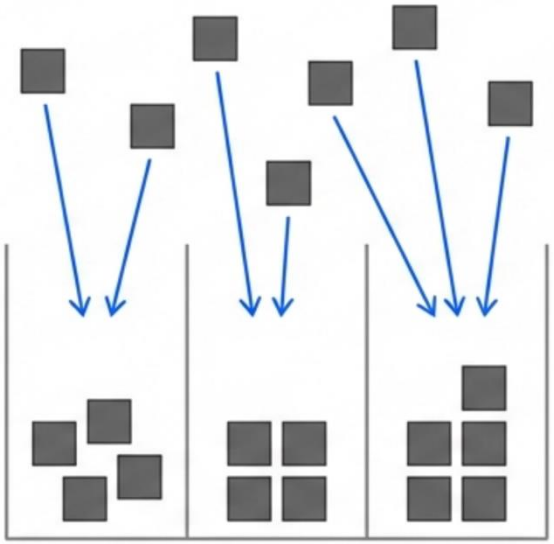
BucketSort

Ordenamiento por Comparación



- Algoritmos como QuickSort o MergeSort se basan en comparar pares de elementos.
- Su límite matemático irreducible es $\Omega(n \log n)$.

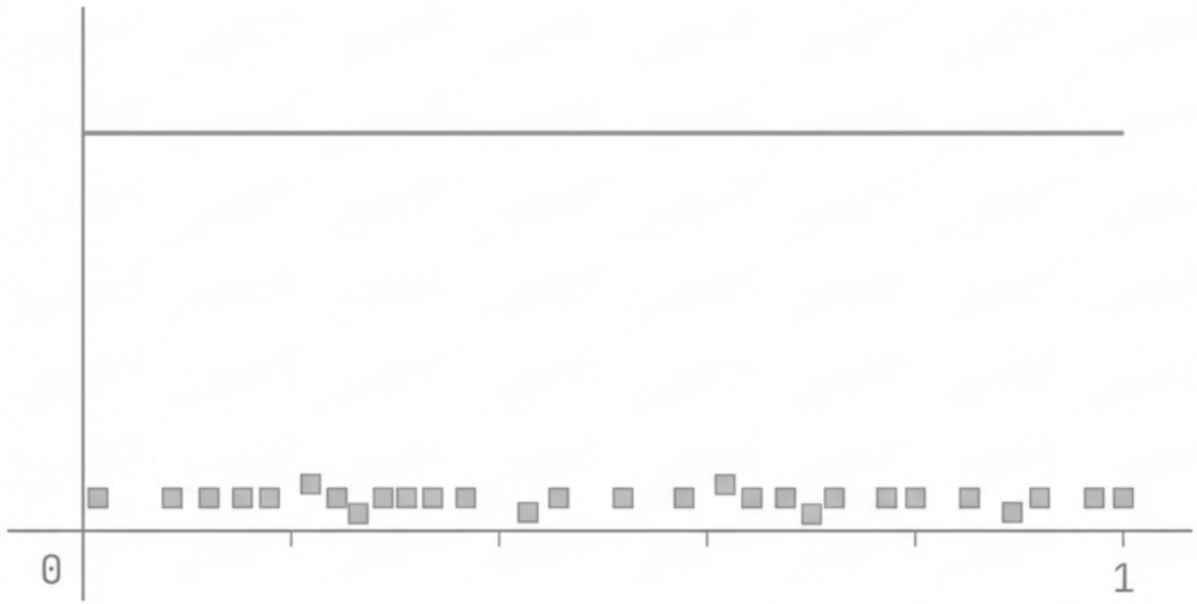
Ordenamiento por Distribución (Bucketsort)



- Utiliza operaciones matemáticas para calcular directamente la posición del elemento.
- Logra un rendimiento lineal promedio de $O(n)$ bajo condiciones específicas.

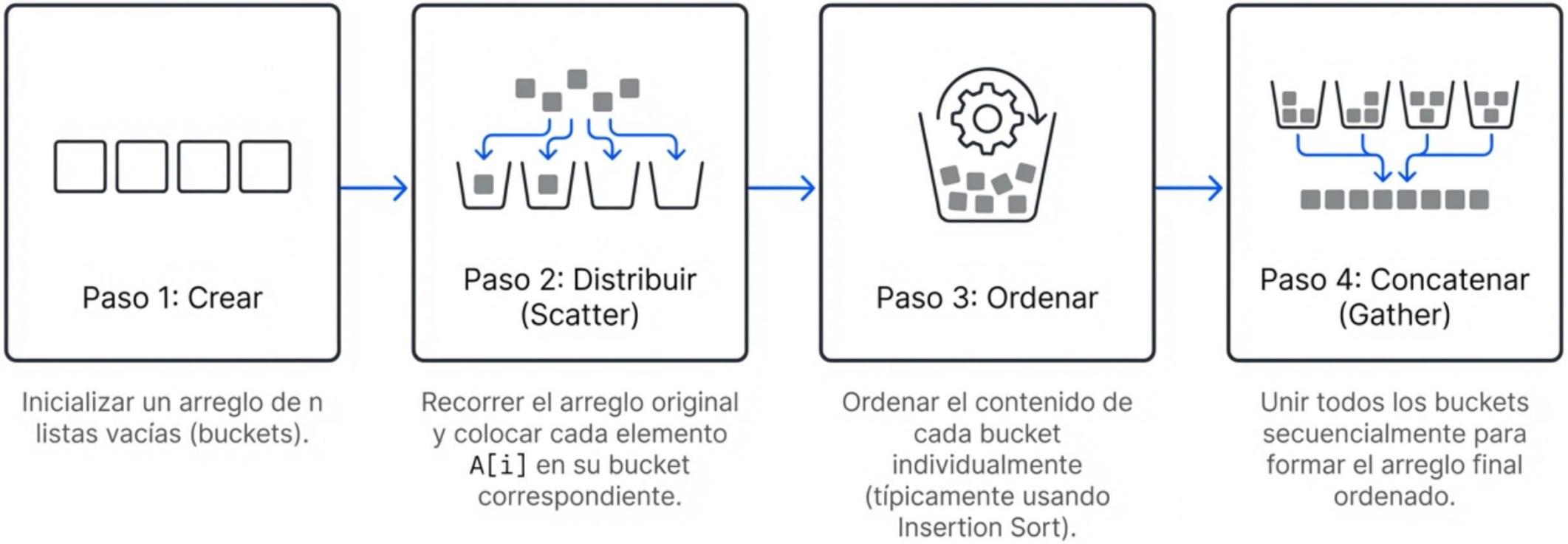
BucketSort

Bucketsort es increíblemente rápido porque asume información a priori sobre los datos de entrada.



Regla de Oro: Los elementos de entrada deben ser variables aleatorias distribuidas de forma uniforme e independiente en el intervalo [0, 1).

BucketSort


Paso 1: Crear
Inicializar un arreglo de n listas vacías (buckets).
Paso 2: Distribuir (Scatter)
Recorrer el arreglo original y colocar cada elemento A[i] en su bucket correspondiente.
Paso 3: Ordenar
Ordenar el contenido de cada bucket individualmente (típicamente usando Insertion Sort).
Paso 4: Concatenar (Gather)
Unir todos los buckets secuencialmente para formar el arreglo final ordenado.

Bucketsort

```
BUCKET-SORT(A, n)
let B[0..n-1] be a new array
for i = 0 to n-1
    make B[i] an empty list
for i = 1 to n
    insert A[i] into list B[floor(n * A[i])]
for i = 0 to n-1
    sort list B[i] with insertion sort
concatenate lists B[0], B[1], …, B[n-1]
return the concatenated lists
```

Paso 1: Inicialización

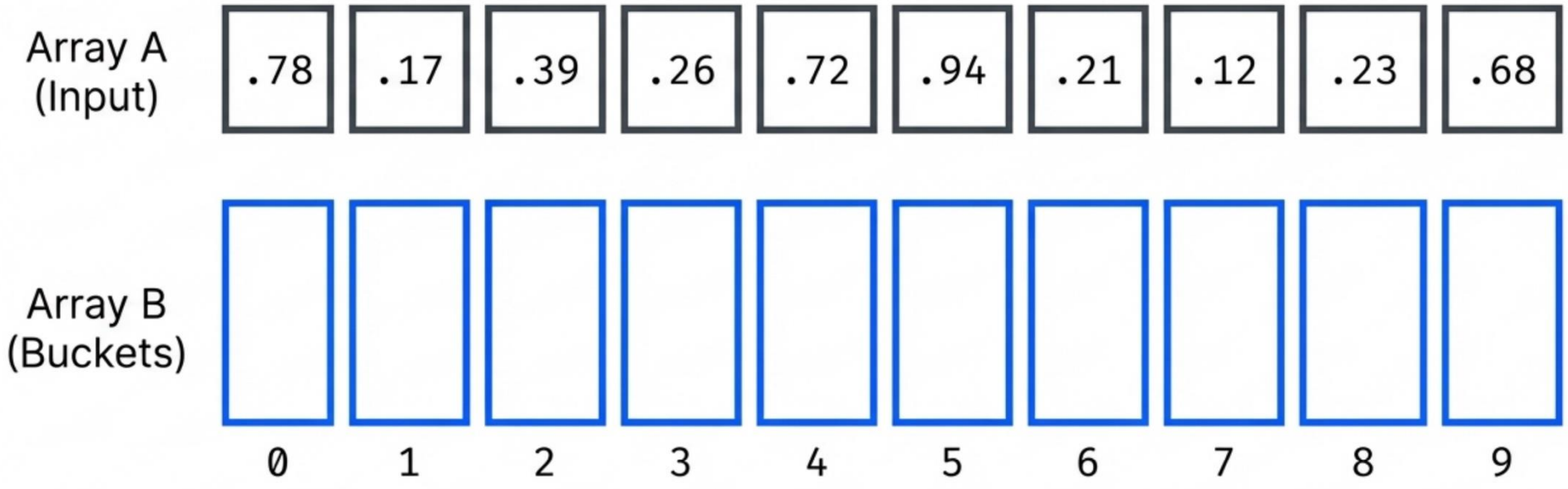
Paso 2: Distribución (Magia del Índice)

Paso 3: Ordenamiento Interno

Paso 4: Ensamblaje

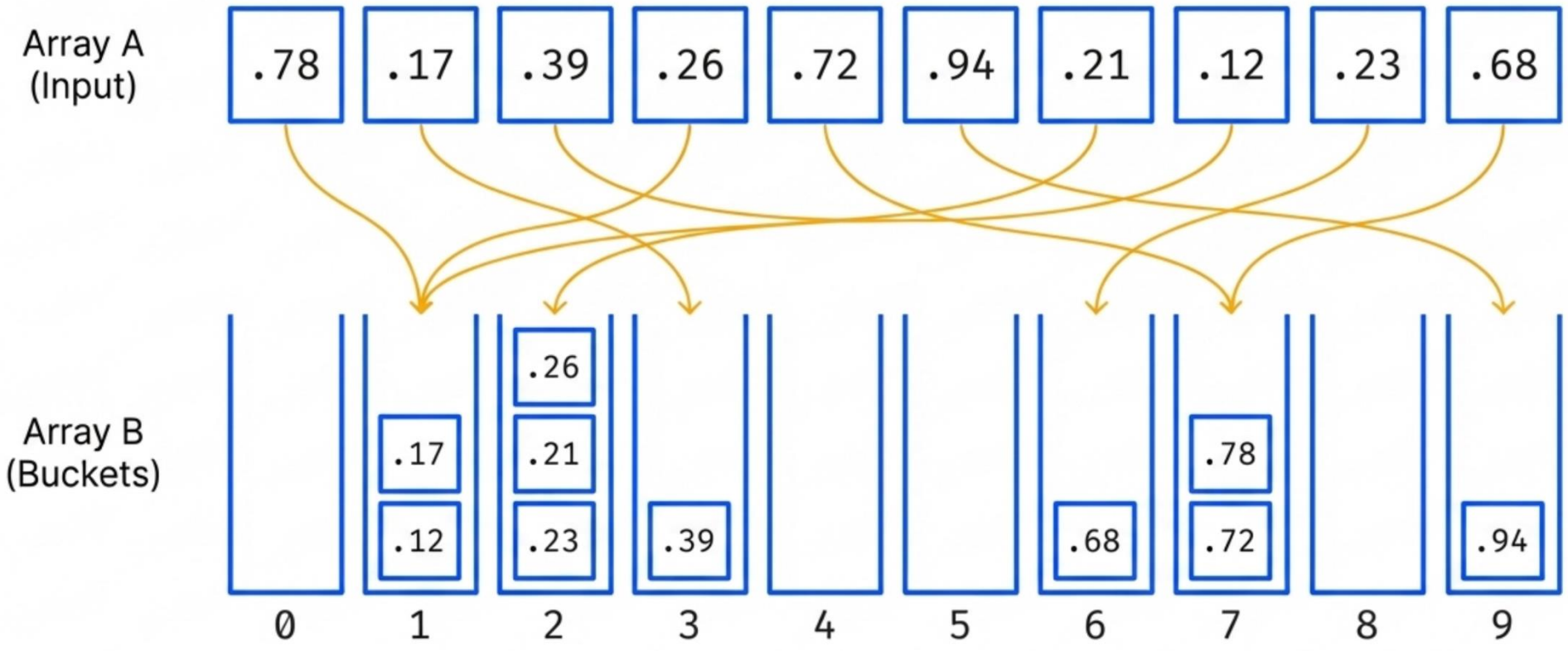
BucketSort

El Estado inicial



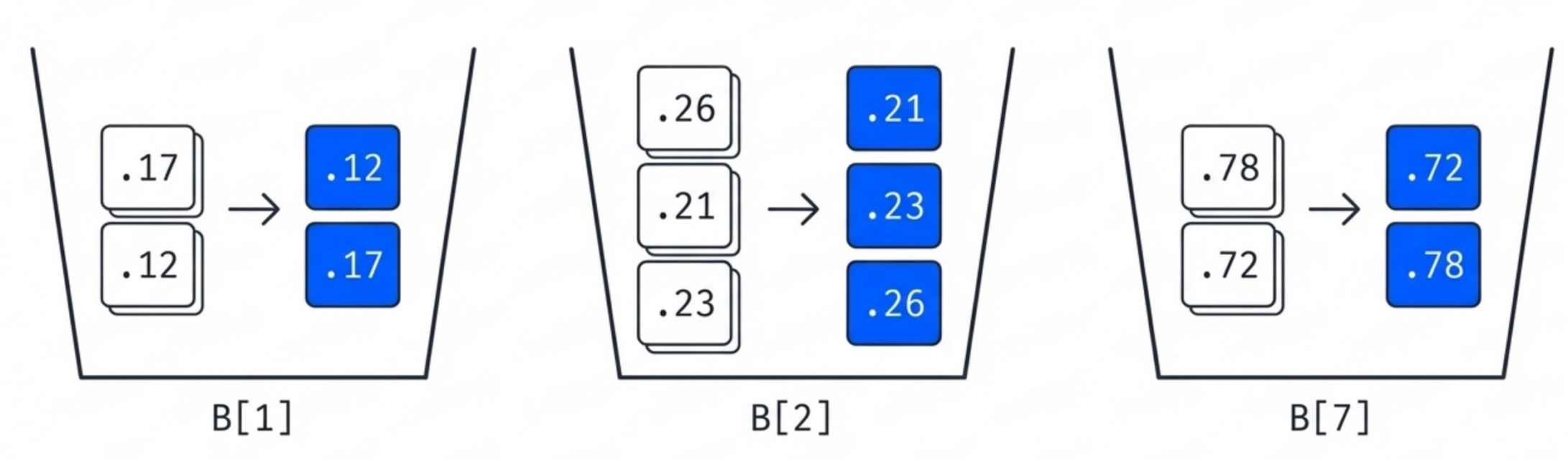
BucketSort

La Fase de dispersión



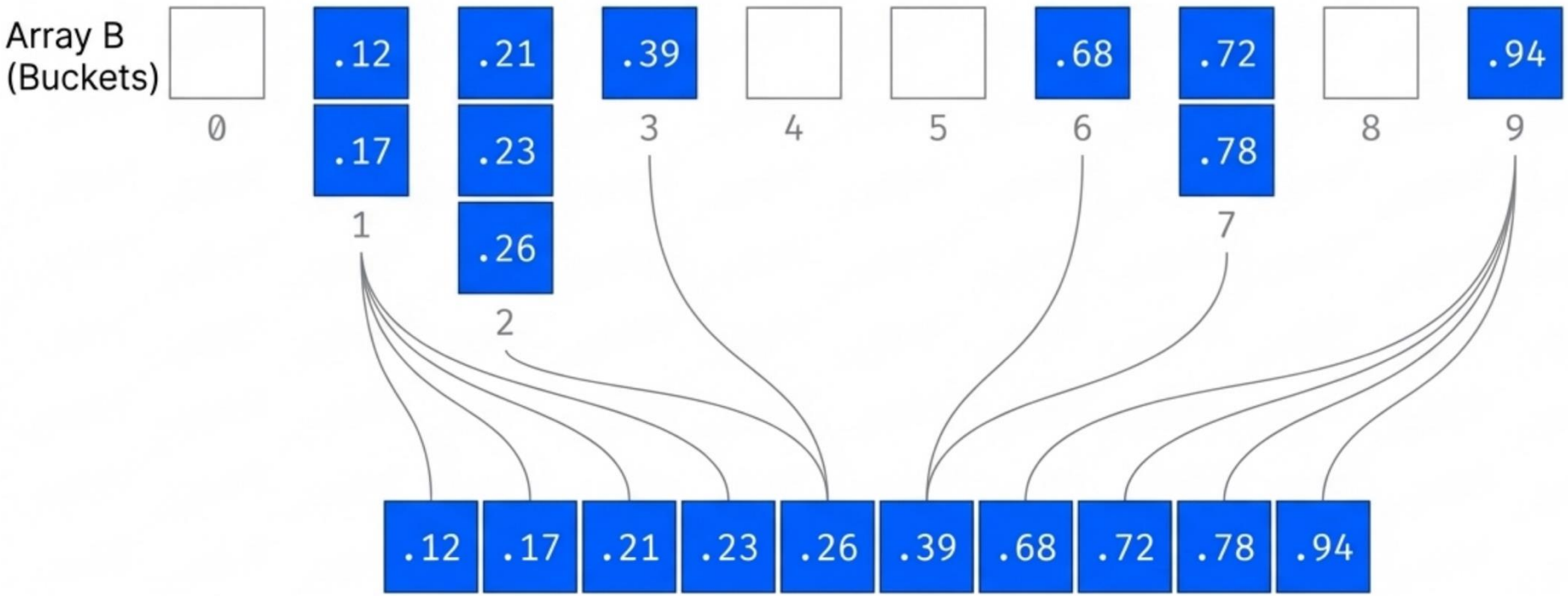
BucketSort

La Fase de ordenamiento interno



BucketSort

La Fase de concatenación (Gather)

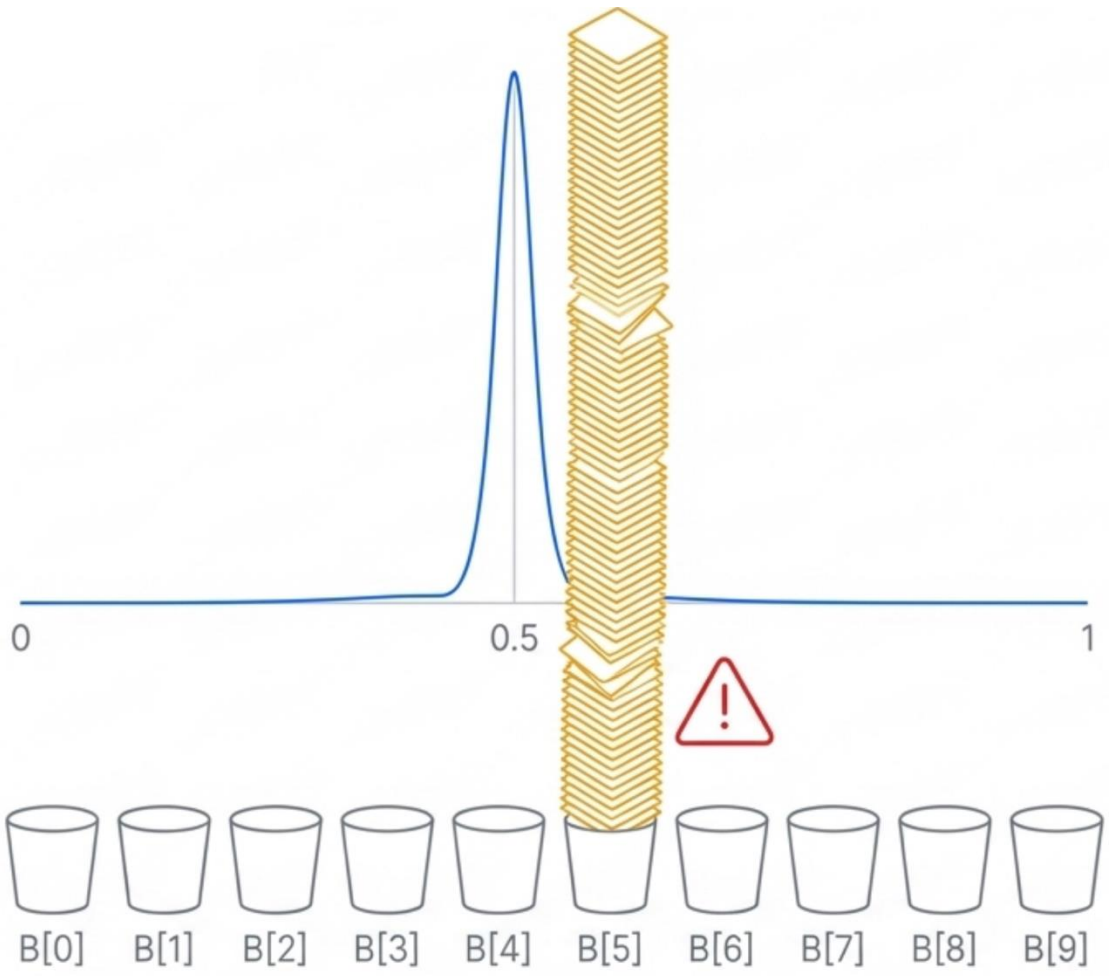


BucketSort

Si la distribución de datos presenta aglomeraciones masivas (clustering), la mayoría o todos los elementos podrían caer en un único bucket.

- La complejidad degenera al tiempo de ejecución del ordenamiento interno del bucket.
- Dado que Insertion Sort toma $O(n^2)$, el tiempo total de Bucketsort en el peor caso se desploma a $O(n^2)$.

Conocer la distribución de sus datos es obligatorio antes de elegir Bucketsort.



BucketSort

Métrica	Valor/Descripción
Tiempo Promedio	$\Theta(n)$ — Lineal. Altamente eficiente.
Tiempo Peor Caso	$O(n^2)$ — Ocurre si todos los elementos caen en un solo bucket.
Complejidad Espacial	$O(n+k)$ — Requiere memoria auxiliar para k buckets. No ordena in-place.
Estabilidad	Sí. Preserva el orden relativo de los elementos duplicados.
Caso de Uso Ideal	Análisis de datos continuos como mediciones de sensores distribuidas uniformemente.